Project Executable Files

Date	3 october 2026
Team ID	SWTID-2026-5338
Project Name	ComicCraft-AI Comic Creator using Gemini Models
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	YES
2	README / Setup Guide (instructions to run the project)	YES
3	requirements.txt / package.json / dependency file	YES
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	YES
6	Deployed application URL (if hosted)	NA
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	YES
10	Demo video or walkthrough (if required)	YES

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

COMICCRAFT – PROJECT FILE STRUCTURE

1. Application Source Code

`app/`

- `main.py` – FastAPI application entry point
- `routes.py` – Application routes and API endpoints
- `gemini_flash.py` – Gemini AI story generation
- `gemini_pro.py` – AI story processing
- `image_generator.py` – AI comic image generation
- `layout_builder.py` – Comic panel layout creation
- `exporters.py` – PDF export functionality

2. Frontend Templates

`templates/`

- `index.html` – ComicCraft home page
- `comic_preview.html` – Generated comic preview
- `export_success.html` – PDF export success page
- `test_image.html` – Image generation test page

3. Static Files

`static/`

- `css/style.css` – Website styling
- `panels/` – Generated comic panel images
- `exports/` – Generated PDF files

4. Documentation

`docs/`

- `MODEL_SELECTION.md` – AI model selection details
- `ARCHITECTURE.md` – Project architecture
- `TESTING.md` – Testing details
- `SKILLWALLET_MAPPING.md` – SkillWallet mapping

5. Project Configuration Files

- `.env.example` – Environment variable template
- `.gitignore` – Git ignored files
- `requirements.txt` – Python dependencies
- `README.md` – Project setup and run instructions

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not deployed. The application runs locally using FastAPI and Uvicorn
Login Credentials (Demo)	Not required
Platform / Hosting Provider	Local Development Environment – FastAPI + Uvicorn
Repository Link	https://github.com/vasilabanuvasilabanu/ComicCraft-AI-Comic-Story-Creator
Demo Video Link	https://drive.google.com/file/d/17raxT-dcsjABzEFv2fIiEY1DHoyCl3gB/view?usp=drivesdk

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

- Windows operating system
- Python 3.x
- Visual Studio Code
- Internet connection
- Gemini API key
- Hugging Face API key

Step 1: Open the ComicCraft_Full_Project folder in Visual Studio Code.

Step 2: Open the VS Code Terminal inside the project folder.

Step 3: Run the application using:
..\env\Scripts\python.exe -m uvicorn app.main:app --reload

Step 4: Open the browser and visit:
http://127.0.0.1:8000

Step 5: Enter the story prompt, character name, setting, story tone and art style, then click Generate My Comic.

Step 6: View the generated five-panel comic and download the generated PDF.

Step 7: FastAPI documentation:
http://127.0.0.1:8000/docs

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	The application currently runs locally and is not hosted online.	Run the application using FastAPI and Uvicorn on the local system.
2	AI story and image generation require valid API keys and internet connection.	Configure `GEMINI_API_KEY` and `HF_API_KEY` in the `.env` file.
3	AI image generation depends on Hugging Face service availability and usage limits.	Ensure the Hugging Face token has the required permissions and retry if the service is temporarily unavailable.